IBM Bob / VS Code

Bob 3拡張機能

構成・実現のしくみ・活用

利用者と開発者のための機能ガイド

ワークフロー管理　　**Bazaarレビュー**　　**コード・文書整合**

2026年9月18日 / 基準 commit 3567883d

3拡張でできること

作業の進行管理と、2種類のレビュー支援

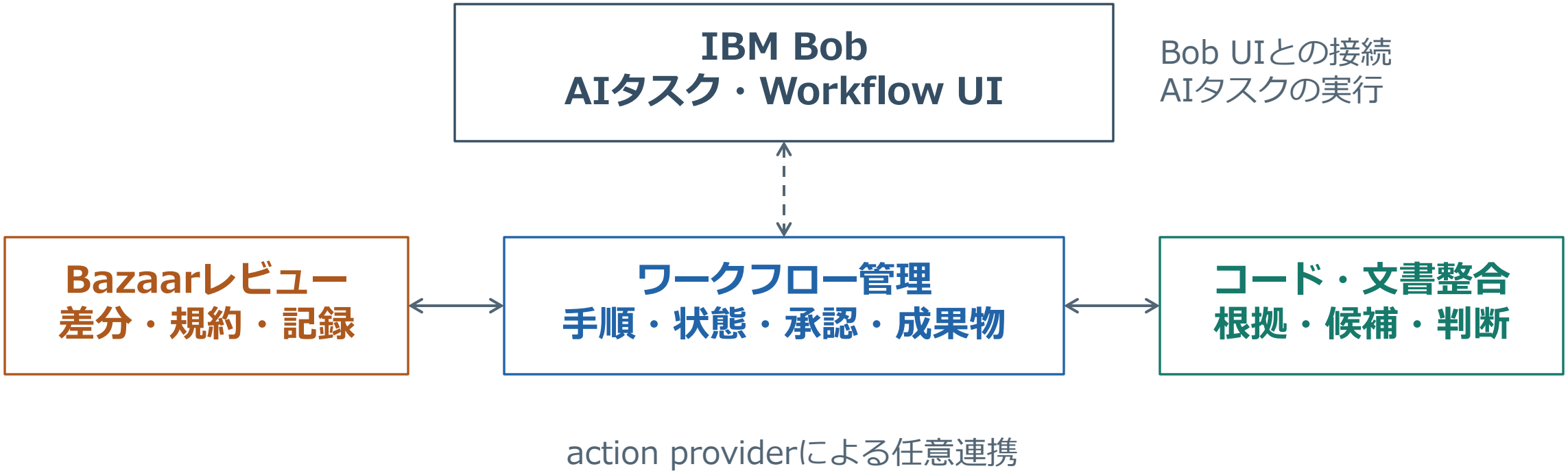
拡張	できること	主な成果物
ワークフロー管理	手順を定義 実行・承認・再開	ワークフロー定義 実行状態・成果物
Bazaarレビュー	変更差分を 規約に沿ってレビュー	レビュー資料 指摘・判断・実績
コード・文書整合	要求・設計・テストと コードを照合	根拠付きレビュー資料 指摘・対応関係

目的に応じて単体利用し、必要な処理をワークフローへ組み込む

全体像

IBM Bobと3拡張の全体構成

共通の実行基盤に、レビュー処理を任意で接続する

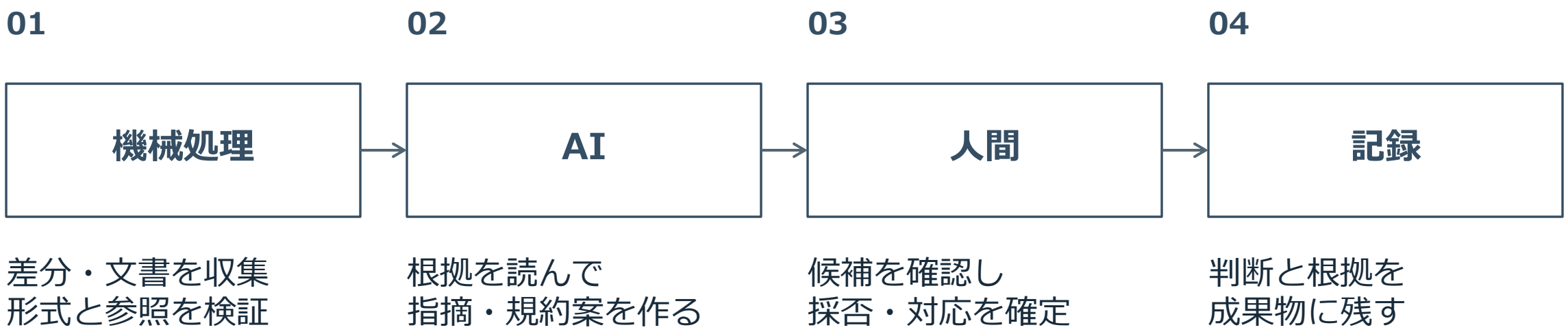


BobによるAI実行と、各拡張のローカル処理を分けて理解する

全体像

機械処理・AI・人間の役割

根拠の整理から、判断の確定までを分担する



形式検証の成功と、内容の正しさ・最終承認は別の確認

5つの公開コマンドと操作画面

通常の操作は、目的別の入口から始める

Command Paletteの表示名	用途
Bob: 操作ハブを開く	ワークフロー選択・実行状況・再開
Bob Workflow: ワークフローを作成・編集	定義の作成・編集 テンプレートの利用
Bob Bazaar Review: レビューを開始	Bazaarレビューの準備から実績確認
Bob Bazaar Review: 資料からプロジェクト規約を作成	資料を使う規約作成の準備
Bob Code Consistency Review: 整合プレレビューを開始	入力から指摘の振り分けまで

内部連携用コマンドは、公開入口と区別して扱う

ワークフロー管理の用途と操作ハブ

繰り返す作業を、状態と成果物を持つ手順にする

手順を選んで実行

カタログから対象を選択
入力値と実行条件を確認
現在のステップを把握

途中で判断・復旧

承認、差し戻し、再試行
停止後の再開
成果物と診断の確認

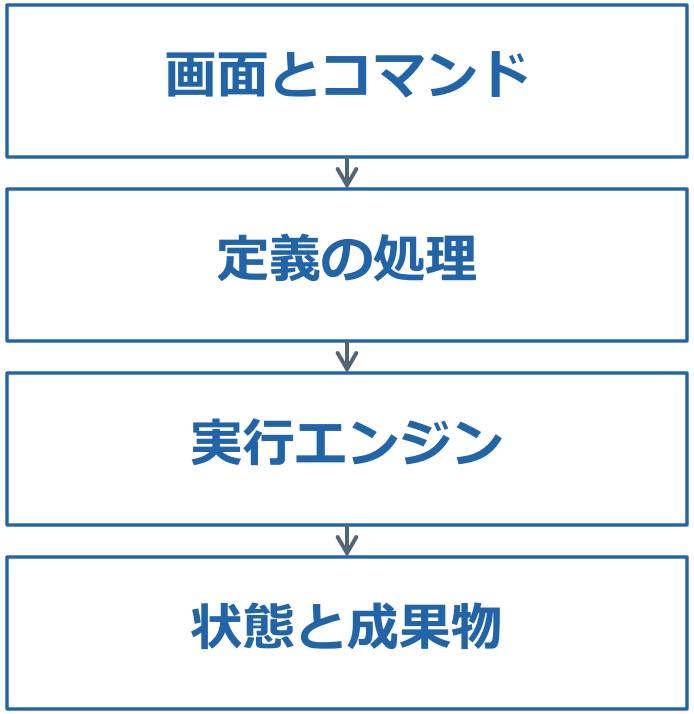
履歴を管理

実行履歴と保存容量の確認
保護する履歴をpin
候補を確認して手動整理

「Bob 操作ハブ」で、実行の状況と次の操作を確認する

ワークフロー管理の内部構成

定義の読み込み、実行、保存、画面表示を分担する



操作ハブ／Builder／Bob UIから操作を受ける

WORKFLOW.mdを解析・検証し、実行用モデルへ変換

providerを呼び出し、承認・分岐・結果保存を制御

run.json／control.json／成果物・snapshotを保存

画面は状態を表示し、エンジンは手順を進め、storeが状態を保持する

WORKFLOW.mdと入力定義

YAMLで手順を定義し、Markdownで説明を添える

```yaml
schemaVersion: workflow-register/v1
name: sample-review
inputs:
  target: { type: string, title: 対象 }
steps:
  - id: analyze
    title: 分析
    type: agent
    prompt: "{{inputs.target}} を要約"
    resultKey: analysis
```

配置

.bob/workflows/
<名前>/WORKFLOW.md

入力と状態

inputs は利用者の指定
state は前段の処理結果

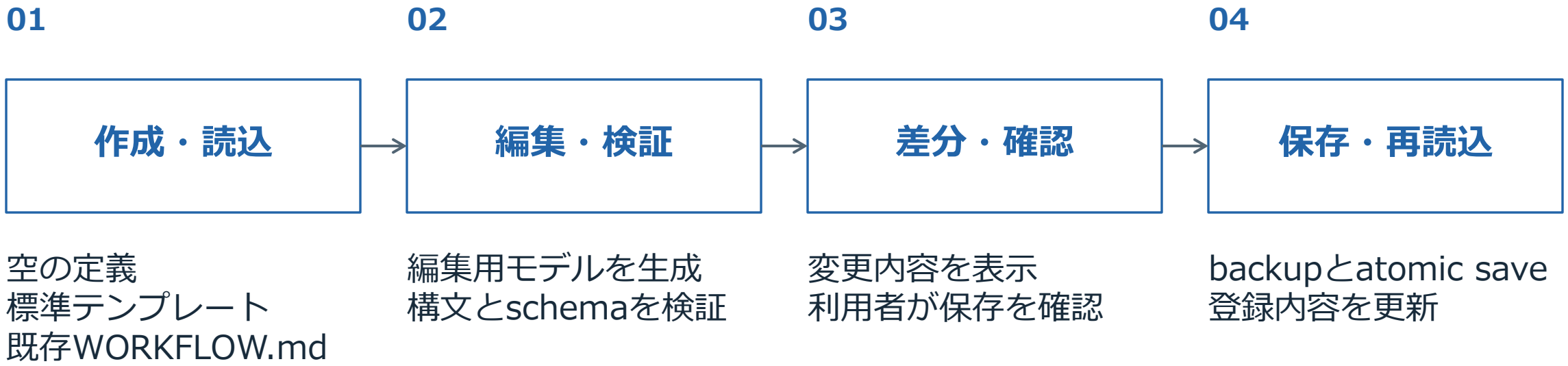
実行前の検証

型・参照・順序・実行条件
を確認する

入力を変えて同じ手順を再利用できる

作成・編集から保存まで

Builderで編集し、差分と検証結果を確認して保存する



既存ファイルとの差分を見てから、保存を確定する

標準テンプレートとAI作成支援

共通手順をプロジェクトに合わせ、同じ保存経路で反映する

標準テンプレート

基本手順にproject profileとcustomizationを適用
生成元の版・hashを記録

AIによる補助

ワークフロー案の作成
既存定義の改善案
診断内容の説明

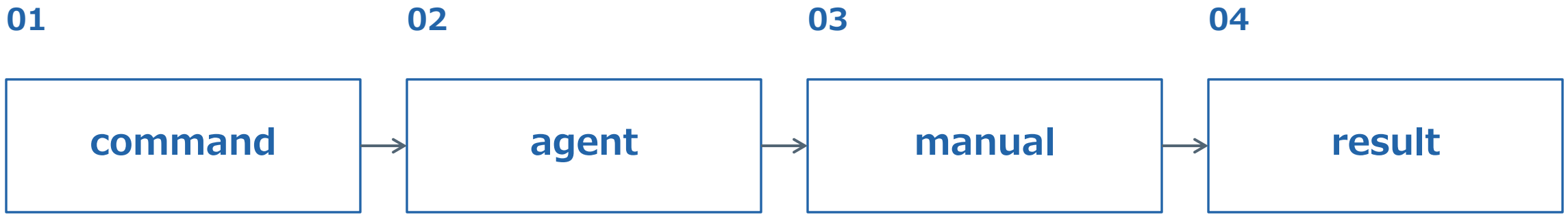
保存前の確認

Preview・Diagnostics・Diff
変更できる範囲の検証
利用者が確認して保存

AI補助には設定済みproviderが必要。生成内容も通常の検証を通す

4種類のステップとデータ受け渡し

機械処理、AI、人の確認、出力を組み合わせる



図は代表的な組み合わせ例。順序はワークフロー定義で決める

実行状態はrun.jsonに保存する

画面やBobのTodoは、保存済み状態をもとに表示する

run.json

定義・入力・step状態・試行履歴
結果・分岐・成果物の由来

↓

操作ハブ / Bob Todoに状態を表示

運用を支える別ファイル

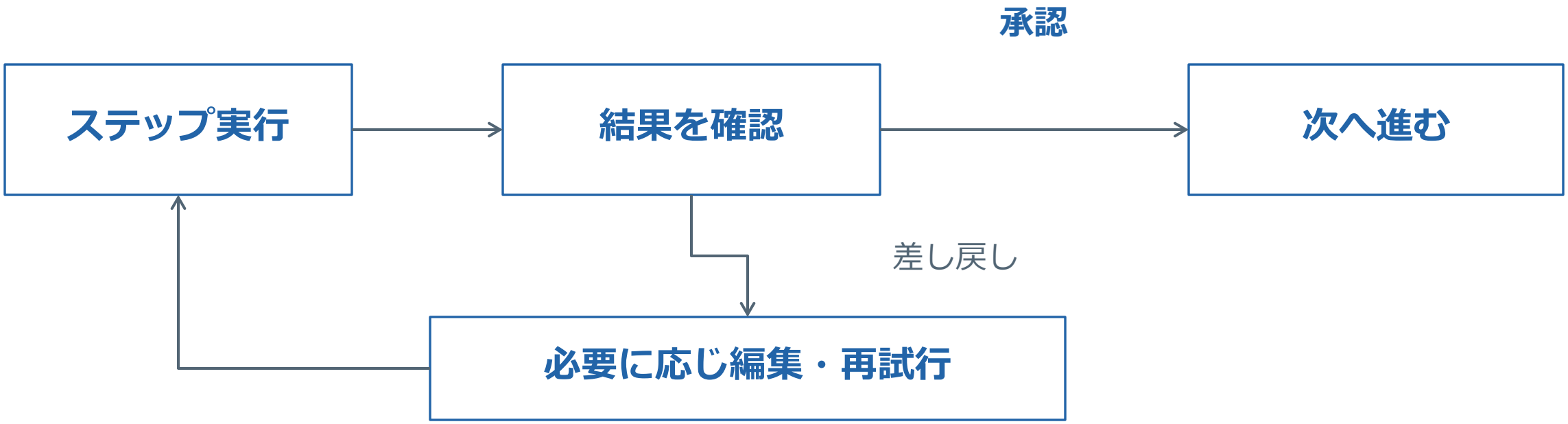
control.json
停止要求・再開メモ

task snapshots
復旧・診断の補助証跡

再開時は保存状態に加え、定義・入力・実行対象の整合も確認する

承認・分岐・再試行

ステップの結果を確認し、次へ進むかやり直すかを決める



ステップ後の確認と、明示的なmanual承認を用途に応じて使い分ける

停止・再開と日常の運用

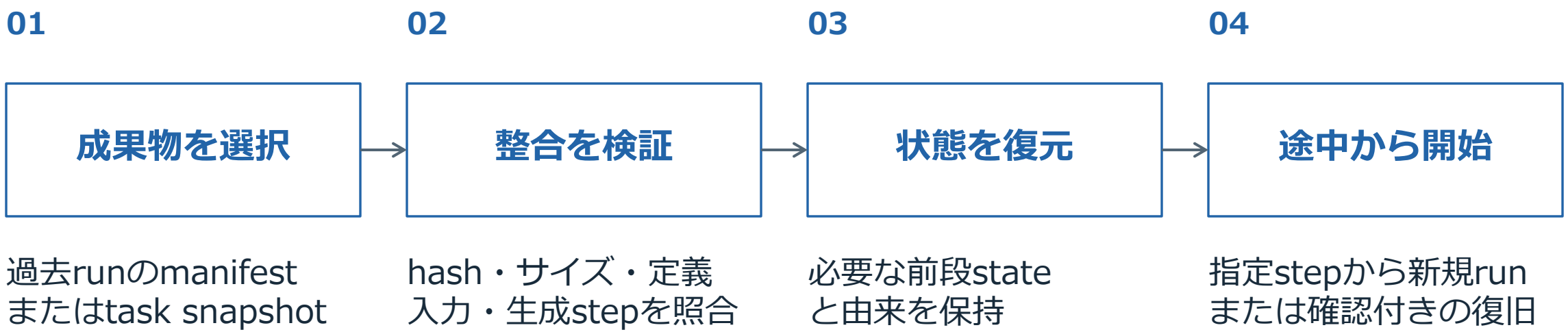
安全な区切りで停止し、状態を確認して再開する

操作	しくみ	利用時のポイント
一時停止	現在stepの後／次のAI呼出し前 時間や呼出し回数でも判定	実行中の処理を強制停止しない
再開・診断	状態・定義・revisionを再確認 診断結果とエラーを表示	停止理由と必要な入力を確認
履歴整理	保持日数・件数・容量から候補作成 preview後に手動適用	実行中・復旧可能・参照中を保護

停止意図、実行状態、履歴整理は別の仕組みとして管理する

成果物の再利用と中断後の復旧

保存済みの結果を検証し、使える範囲だけ引き継ぐ



snapshotがあるだけでは、自動的に完了や再開とは判定しない

機械チェックと工程別ワークフロー

ローカルの確認処理とレビューを、工程の記録につなぐ

機械チェック

登録したprofileでローカル処理を実行
標準出力・ログ・判定を保存
AIレビューの前段で利用

工程別の手順

要求分析・設計・コード・テスト
工程ごとの入力と根拠を収集
承認後に工程記録へ保存

チェック結果とAIレビュー、人間の判断を別々の根拠として残す

Bazaarレビューの流れ

対象の選択から、人の判断と実績確認まで

01	02	03	04	05
対象を選ぶ	**資料を作る**	**Bobで確認**	**結果を確認**	**実績を確認**
単一revision 範囲・作業ツリー から選択	差分・履歴 変更ファイル 規約をまとめる	資料を渡し 指摘と根拠を 得る	検証して保存 人が指摘を 振り分ける	記録を確定 キャンペーン 単位で集計

差分の取得、AIレビュー、人の判断、記録を同じ流れで扱う

Bazaarレビューの内部構成

共通の処理をGUI・ワークフロー・MCPから利用

公開入口と画面

レビューGUI／規約作成GUI
利用者の入力と進行を扱う

↓

処理をつなぐ層

Review Run／ワークフロー連携
対象・結果・進捗を引き継ぐ

↓

レビューの共通処理

Bazaar操作／規約／結果検証／記録
資料抽出と規約反映

↓

外部との接続

Bazaar CLI／Bob context／MCP
各接続先の条件を検証

入口が異なっても、対象の解決・検証・保存の処理を共有する

差分の収集とレビュー資料の生成

対象情報をそろえ、保存してからBobへ渡す

01	02	03	04
対象を確定	**情報を収集**	**資料を生成**	**保存して渡す**
例：revision 120～125を指定	CLIで差分を収集 履歴・変更一覧も取得	対象情報・追加資料 規約をMarkdown 形式で集約	レビュー資料を保存 Bob contextへ追加

どの変更を、どの規約で確認するかをレビュー資料に残す

プロジェクト規約による確認観点

確認項目・根拠・判定形式をレビュー資料に添える

確認項目を定義

checklistで規約を指定
例：RTスレッドのI/O禁止
外部I/F構造体の互換性

根拠と結び付ける

pass/failには根拠が必要
証拠不足はunknown
failには規約ID付きの指摘

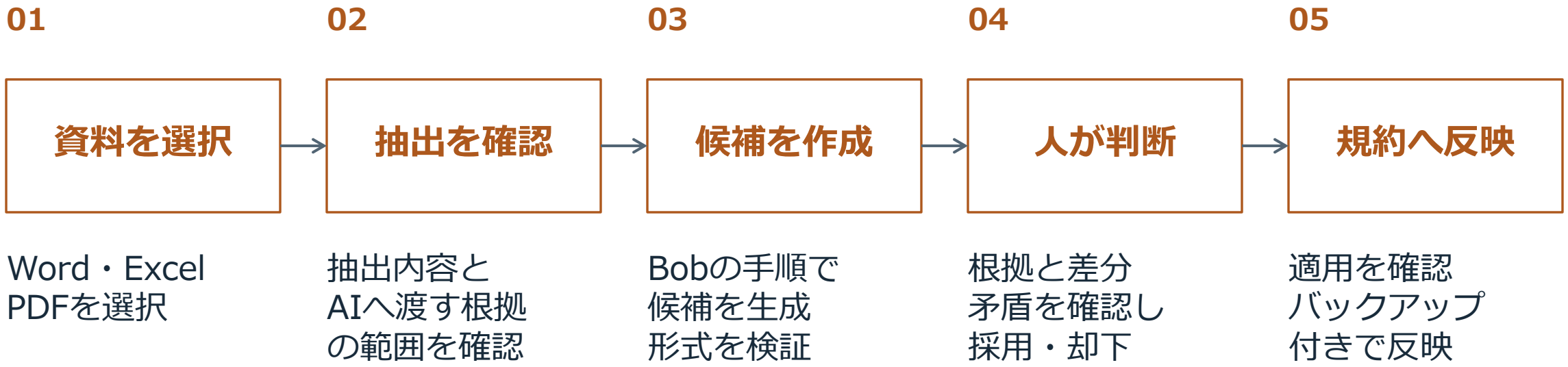
結果をそろえる

schemaに沿うJSONと要約
承認済みのプロジェクト情報
追加のレビュー指示も利用

規約を文章で添えるだけでなく、IDと根拠を持つ結果へつなぐ

資料からのプロジェクト規約作成

抽出した根拠を使い、人が確認して正式規約へ反映



資料の本文、AIの候補、人の判断、正式規約を段階で分ける

結果を検証し、人が採否を決める

AI出力を保存前に検証し、判断を別の記録に残す

人の判断	必要な入力	記録の意味
採用	担当者または対応	対応すべき指摘として扱う
却下	理由	採用しない理由を残す
要調査	担当者と対応	追加確認の担当と作業を決める
保留	理由	今すぐ判断・対応しない理由を残す

AIの指摘は提案として受け取り、採否の正本は人が保存する

Review Runとレビュー実績

途中再開の情報と、確定した成果物を分けて保存

進捗の記録

review-run.json
対象・レビューID・段階・成果物の位置

↓

判断と結果の記録

結果JSON／triage.yaml／record.yaml
AI出力、人の判断、確定した実績

↓

キャンペーン集計

採否・規約別傾向・所要時間を集計
推計削減時間＝基準−Bob−人の判断

再開できる進捗と確定記録を残し、効果は推計値として比較する

MCP連携と利用条件

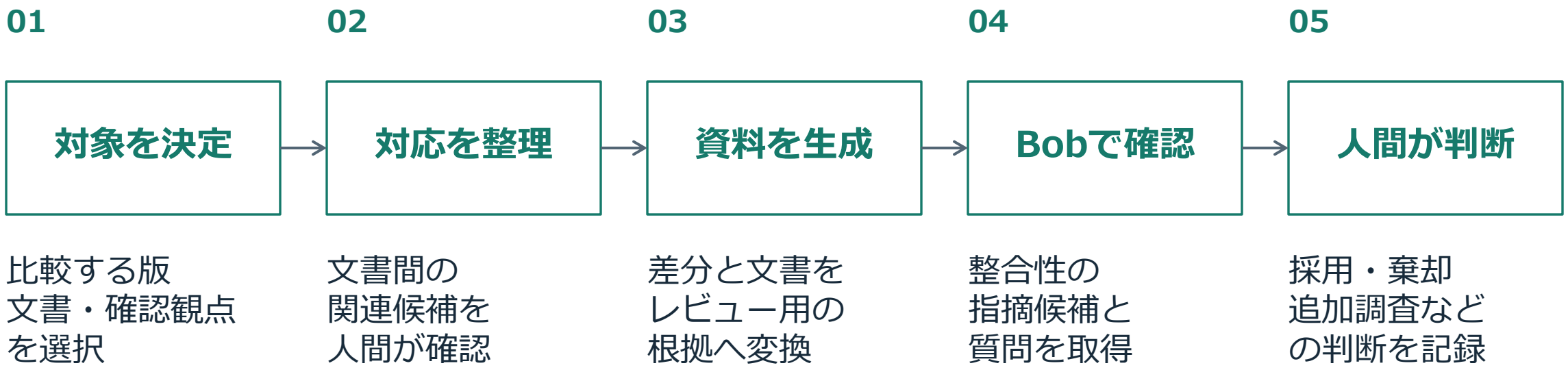
読み取りを基本に、必要な外部機能と許可範囲を設定

機能	利用条件	処理の範囲
Bazaar取得	Bazaar CLIが必要	差分・履歴・状態・ファイル参照
Bob連携	Bob拡張と利用するモデル設定	資料をcontextへ渡しレビュー
規約作成	対応資料とワークフロー連携	PDFはpdftotextが必要
MCP参照	許可rootの設定が必要	Bazaar・規約・保存済み結果
MCP初期化	書き込みの明示有効化が必要	project_rules_initのみ

MCPは既定で読み取り専用。BazaarへのcommitなどはBazaarへのcommitなどは公開しない

整合プレレビューの流れ

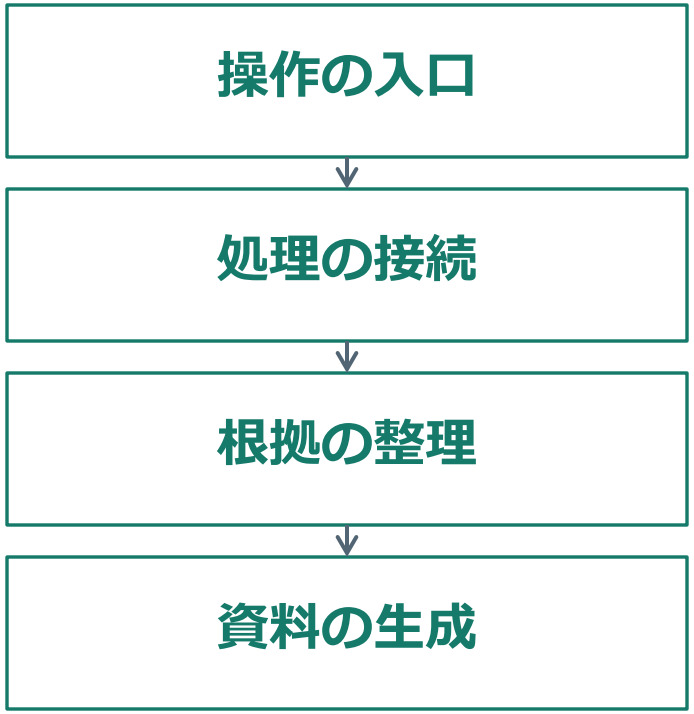
要求・設計・テストとコード変更のずれを探す



根拠の準備から指摘の振り分けまで、同じ画面で進める

整合プレレビューの内部構成

画面とworkflowが同じ処理を呼び出す



主画面と内部コマンド
workflowからも操作を呼び出せる

入力検証、差分収集、文書・コード解析を接続

範囲・量の計画、対応候補、生成物台帳を管理

review-packageとBob投入用テキストを生成

画面処理とレビュー処理を分け、入口が変わっても同じサービスを使う

条件・差分・文書を入力する

比較範囲と使う資料を明示してレビューを始める

入力	指定する内容	扱い
レビュー条件	ID・変更種別・確認観点	review-input.yamlに保存
コード差分	Git / Bazaar、base / head	比較する版を明示
対象文書	Markdown / DOCX / XLSX	節・ケース・行・シートを選ぶ
文字コード	UTF-8 / Shift-JIS系	設定と自動判定で読み込む
読み取り上限	文書1件5MiB、Bob入力2MiB	既定値。設定で調整可能

選んだ比較範囲と文書が、レビュー根拠の出発点になる

コード根拠と資料量の管理

コード解析、リポジトリ索引、生成物台帳を組み合わせる

コード根拠

C/C++は関数と
呼出候補を抽出
その他は差分行を根拠化

範囲と量の計画

必要に応じて
リポジトリ索引を利用
優先度と推定量をもとに
採用・除外を記録

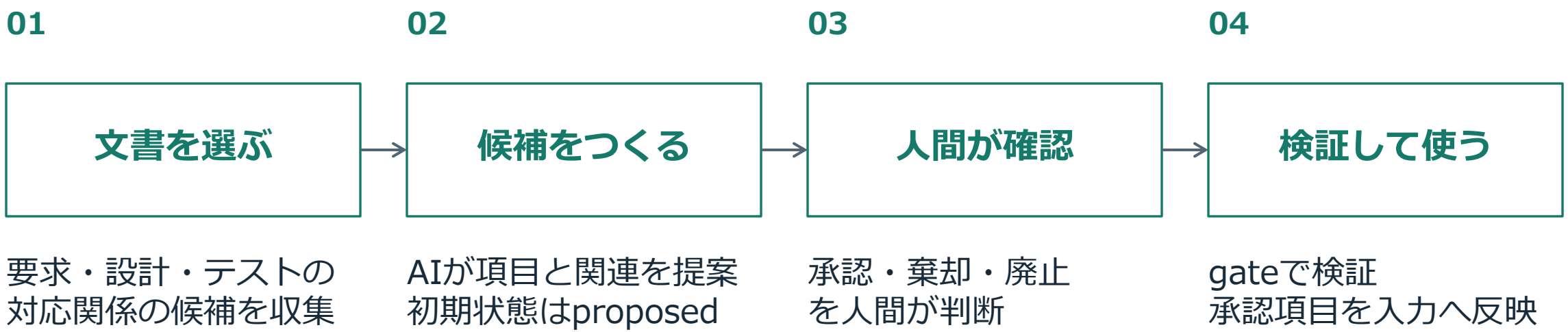
生成物の台帳

入力・内容のhashと
依存関係を記録
最新・古い・欠落を区別

根拠の選定計画を記録し、実際の投入量には別途上限をかける

文書間のトレーサビリティ

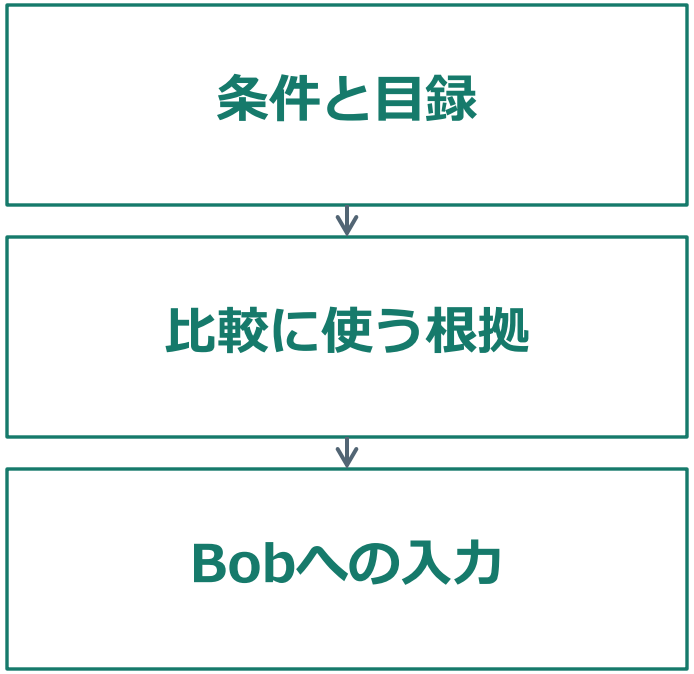
元文書を変えず、候補と人間の承認を残す



AIの候補と人間の確定判断を、元文書とは別に保持する

Bobへ渡すレビュー資料

何を比較し、どの根拠を渡したかを残す



入力条件・変更ファイル・根拠IDの一覧
manifestに生成情報と入力ハッシュ

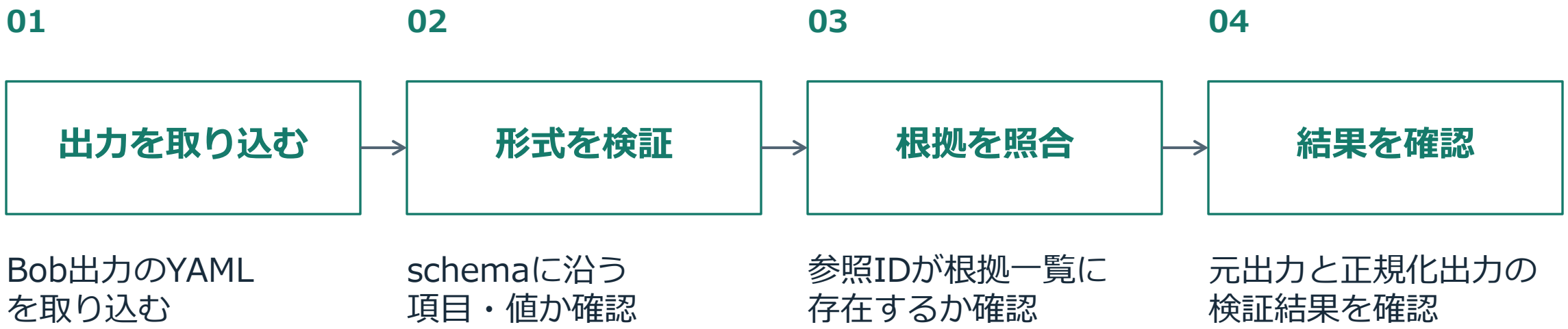
文書抜粋・差分・コード断片・対応候補
抽出や切り詰めの警告も記録

promptと根拠をbob-input.mdへ集約
生成した内容をプレビューして確認

投入内容と生成情報をファイルで残し、後から参照できるようにする

Bob出力の形式・根拠ID検証

検証結果を確認して、指摘の判断へ進む



機械検証は形式と参照の確認。指摘内容の妥当性は人間が判断する

指摘の判断と引き継ぎ資料

出力の写しと判断を分け、一覧を再生成する

元出力の写し

Bob由来のsourceを保存
後から判断の対象を追跡

人間の判断

採用・棄却・追加調査
保留・質問を選択
理由・担当者・期限を記録

引き継ぎ資料

指摘・質問・後続作業
を一覧化
正式レビューや
修正作業へ渡す

AIの指摘候補に、人間の理由と担当を加えて次の作業につなぐ

具体例：タイムアウト時の戻り値

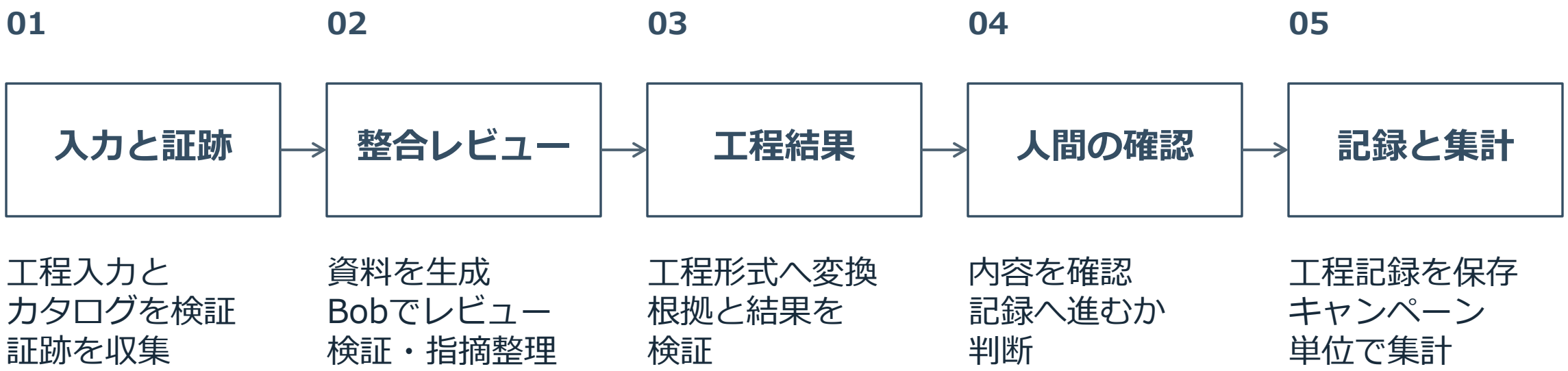
同梱サンプルを用いた説明例：要求・設計・テストと戻り値を照合

根拠	timeout発生時の内容	読み取れること
要求 REQ-123	ERR_TIMEOUTを返す	異常終了を上位へ伝える
詳細設計 DD-88	カウンタ加算＋ERR_TIMEOUT	処理と戻り値を規定
変更後コード	カウンタ加算＋ERR_OK	戻り値が文書と食い違う
テスト TC-789	期待値はERR_TIMEOUT	異常系の期待結果とも食い違う
人間の確認	実装の誤りか、仕様変更か	根拠を示して正式レビューへ

同じ事象の根拠を並べると、修正点と確認質問を具体化できる

コード事前チェックの一連の実行例

ワークフロー管理が整合プレレビューを工程として実行する



同梱のprocess-code-precheckに基づく処理例

拡張間APIと機能追加の仕組み

providerとsinkを登録し、実行基盤から呼び出す

接続点	役割	利用例
registerActionProvider	機械処理の呼び出し口	レビュー資料生成・出力検証
registerAgentProvider	AI実行の呼び出し口	promptとstateを渡して結果取得
registerResultSink	結果の保存・転送先	ファイル保存・domain処理への連携
runWorkflow / runWorkflowStep	定義全体やstepの実行	通常実行・指定step実行

専門処理をレビュー拡張に置き、手順と状態管理は共通基盤が担う

対応形式・依存ツール・解析の限界

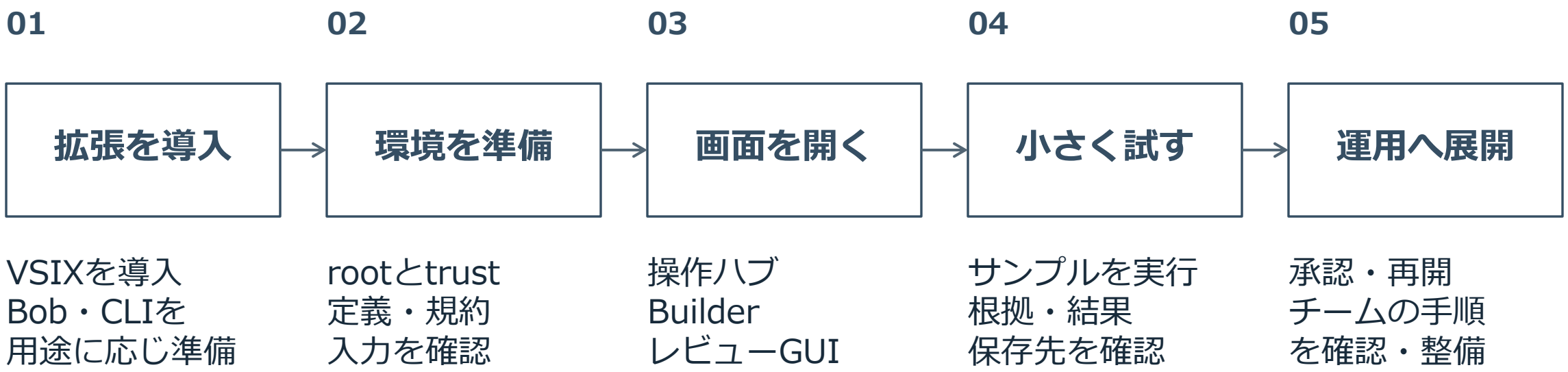
用途ごとに、扱える入力と必要な実行環境を確認する

対象	入力・依存	主な制約
ワークフロー	WORKFLOW.md AI stepはBobまたはprovider	Bobなしでも編集・検証等は可能 AI実行には接続先が必要
Bazaarレビュー	Bazaar CLI 規約作成はDOCX・XLSX・PDF等	PDFはpdftotextと抽出可能な文字 旧Office形式は変換が必要
整合プレレビュー	Git / Bazaar差分 Markdown・DOCX・XLSX	PDFは文書抽出対象外 C/C++は関数・呼出候補の抽出

AIが出した候補は、根拠と実際の仕様に照らして人が確認する

導入と初回利用の手順

対象workspaceとツールを整え、サンプルで出力を確認する



最初は結果を追跡できる小さな対象で、入力から記録までを確認する

用途別の機能選択と参照資料

目的を決め、該当拡張のREADMEから詳しい手順へ進む

手順をそろえたい

ワークフロー管理
作成・実行・承認・再開
参照：ワークフロー管理
README

変更を規約で見たい

Bazaarレビュー
差分・規約・指摘・実績
参照：Bazaarレビュー
README

仕様とのずれを見たい

コード・文書整合
差分・要求・設計・テスト
参照：整合プレレビュー
README

各ページの発表者ノートに、詳しい説明と実装の参照元を収録